

# Image Compression

For this practice we have implemented the following code

```
1 function [UC, sigmaC, VC] = compress(imjpg, p)
2     % Checking input arguments: imjpg must be a string and p
   must be a positive number
3     if not(isreal(p) & (p > 0))
4         disp('THE NUMBER OF SINGULAR VALUES MUST BE POSITIVE');
5         UC = '???';
6         sigmaC = '???';
7         VC = '???';
8         return;
9     end;
10
11     p = int16(p);
12     photo = imread(imjpg, 'jpg');
13     resolution = size(photo);
14
15     % Validating the image resolution
16     if min(resolution(1:2)) < p
17         disp('THE NUMBER OF SINGULAR VALUES FOR COMPRESSION
   MUST BE');
18         disp('COMPATIBLE WITH THE IMAGE RESOLUTION');
19         UC = '???';
20         sigmaC = '???';
21         VC = '???';
22         return;
23     end;
24
25     % Conversion to double for SVD computation
26     f2 = double(photo);
27
28     % SVD computation for each RGB component
29     for k = 1:3
30         [U(:,:,k), sigma(:,:,k), V(:,:,k)] = svd(f2(:,:,k));
31     end;
32
33     % Reconstruction of the compressed image
34     for k = 1:3
35         f2r(:, :, k) = U(:, 1:p, k) * sigma(1:p, 1:p, k) * V(:,
   1:p, k)';
36     end;
37
38     % Displaying the compressed image
```

```

39     image(uint8(round(f2r)));
40
41     % Storing output matrices for future manipulation
42     for j = 1:3
43         for k = 1:p
44             sigmaC(k, j) = sigma(k, k, j);
45         end;
46     end;
47
48     UC = U;
49     VC = V;
50
51     clear k j f2r U V sigma f2;
52 end
53
54
55 % Applying the compress function to 'image_practice23.jpg' with
    dynamically chosen ranks
56
57 % Name of the image file
58 imageName = 'image_practice23.jpg';
59 photo = imread(imageName);
60 resolution = size(photo);
61 max_rank = min(resolution(1:2)); % Determine the maximum rank
    of the image
62
63 % Calculate p values dynamically
64 p_half = round(max_rank / 2);
65 p_tenth = round(max_rank / 10);
66
67 % Moderate rank - around 100 (if applicable, otherwise use 1/2
    rank)
68 if max_rank >= 100
69     p_moderate = 100;
70 else
71     p_moderate = p_half; % If the image is too small, use 1/2
        rank instead
72 end
73
74 % Our choice of p = 20 (if applicable, otherwise use 1/10)
75 if max_rank >= 100
76     our_p = 20;
77 else
78     our_p = p_tenth; % If the image is too small, use 1/2 rank
        instead

```

```

79 end
80
81 % Displaying the original image in a separate figure
82 figure;
83 imshow(photo);
84 title('Original Image');
85
86 % Displaying the compressed images in separate figures
87
88 figure;
89 compress(imageName, p_tenth);
90 title('Compressed Image (Rank ~ 1/10)');
91
92 figure;
93 compress(imageName, p_half);
94 title('Compressed Image (Rank ~ 1/2)');
95
96 figure;
97 compress(imageName, p_moderate);
98 title('Compressed Image (Moderate Rank)');
99
100 figure;
101 compress(imageName, our_p);
102 title('Compressed Image (Our Rank)');

```

We have chosen to apply the code to the following picture:



Figure 1: Original Picture.

For this picture we have obtained the following 4 compressed pictures:

**Compressed Image (Rank ~ 1/10)**



Compressed Image (Rank ~ 1/2)





Compressed Image (Moderate Rank  $p=100$ )



Compressed Image (Our Rank  $p=20$ )





Let's assume the image is represented as a matrix of size  $m \times n$ , and let  $U$ ,  $\Sigma$ , and  $V$  be the SVD factorization of this matrix. By keeping only the first  $p$  singular values (where  $p$  is much smaller than the original rank of the matrix), we can approximate the original matrix while reducing its size.

The amount of information required to store the original image matrix is  $3 \times m \times n$ , where the factor of 3 accounts for the three color channels (RGB). When we compress the image by keeping only the first  $p$  singular values, the storage requirement for each color channel becomes  $m \cdot p + p + n \cdot p$ . This is because:

- $m \cdot p$  is the size of the matrix  $U$ ,
- $p$  is the number of elements on the diagonal of the matrix  $\Sigma$ ,
- $n \cdot p$  is the size of the matrix  $V^T$ .

Thus, the total information required for the compressed image is  $3 \times (m \cdot p + p + n \cdot p)$ .

The compression rate ( $c_r$ ) is then the ratio of the information required to store the compressed image to the information required to store the original image, that is,

$$c_r = \frac{m \cdot p + p + n \cdot p}{m \cdot n}$$

We implement this formula in MATLAB and apply it to our pictures:

```

1 % Function to calculate compression rate based on SVD
2 function cr = calculateCompressionRate(m, n, p)
3     % Information stored in the original image
4     originalInfo = 3 * m * n;
5
6     % Information stored in the compressed image
7     compressedInfo = 3 * (m * p + p + n * p);
8
9     % Compression rate calculation
10    cr = compressedInfo / originalInfo;
11 end
12
13 % Dimensions of the image (example values, replace with actual
14   image dimensions)
15 [m, n, ~] = size(photo);
16
17 % Compute compression rates
18 rate_tenth = calculateCompressionRate(m, n, p_tenth);
19 rate_moderate = calculateCompressionRate(m, n, p_moderate);
20 rate_half = calculateCompressionRate(m, n, p_half);
21 our_rate = calculateCompressionRate(m, n, our_p);
22
23 % Display the compression rates

```

```
23 fprintf('Compression rate for rank ~ 1/10: %.4f\n', rate_tenth)
    ;
24 fprintf('Compression rate for moderate rank: %.4f\n',
    rate_moderate);
25 fprintf('Compression rate for rank ~ 1/2: %.4f\n', rate_half);
26 fprintf('Compression rate for our rank: %.4f\n', our_rate);
```

Obtaining the following compressed rates:

```
1 Compression rate for rank ~ 1/10: 0.1713
2 Compression rate for moderate rank: 0.2063
3 Compression rate for rank ~ 1/2: 0.8543
4 Compression rate for our rank: 0.0413
```